

Bancos de Dados Chave- Valor

Banco de Dados II

Rafael Escalfoni





CONSISTÊNCIA VERSUS DISPONIBILIDADE E OS BANCOS CHAVE VALOR

- Alta disponibilidade (High Availability)
 - Manter o serviço funcionando pelo maior tempo possível, sem quedas.
- Motivos de quedas:
 - Falha de software, falha de hardware, queda de rede, energia, falha humana
- Como resolver
 - Manter vários servidores replicando os dados, de preferência em lugares diferentes
 - Mesmo usando cloud computing, os provedores do serviço costumam oferecer opções de regiões para distribuir os servidores virtuais em diferentes data centers



PROBLEMA...

- Se tivermos mais de um servidor de banco de dados, como assegurar que todos possuem os mesmos dados replicados **a todo e qualquer instante???**
- Os servidores vão ler e escrever, compartilhar dados entre si e pode acontecer de perderem a comunicação ou parar de responder em algum momento...



CAP Theorem

- Consistência (**C**onsistency), Disponibilidade (**A**vailability) e Tolerância a particionamento (**P**artition-tolerance)
 - Itens desejados mas impossível garantir todos simultaneamente...



AUMENTANDO A DISPONIBILIDADE

- Estratégia Master/Slave
 - Criamos um servidor principal (máster) e um outro secundário com uma cópia de dados (Slave)
 - As escritas são feitas exclusivamente no **master**, que irá replicar os dados no slave
 - Mesmo nesta abordagem, já temos decisões que podem impactar na consistência, disponibilidade ou tolerância ao particionamento
 - Se o master confirmar a escrita antes de propagar para o slave, uma consulta for realizada, teremos dados desatualizados. (stale)



AUMENTANDO A CONSISTÊNCIA

- Em uma estratégia Master Slave, podemos usar o conceito de transação distribuída, onde só validamos os dados após todas as réplicas confirmarem a escrita.
- PORÉM...
 - A operação de escrita ficará bem mais LENTA
 - Se ocorrer uma falha de comunicação entre servidores, não poderemos confirmar nenhuma escrita



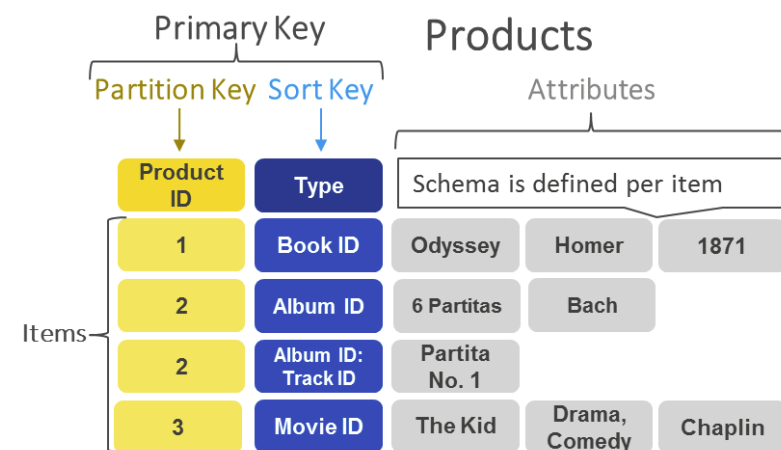
Bancos NoSQL e Arquitetura de Replicação *Masterless*

- Sem Master...
 - Cada servidor é um nó, podendo receber requisições de leitura ou escrita
 - Mesmo com falhas de rede, o *cluster* (conjunto de nós) consegue manter as leituras e escritas disponíveis
 - A consistência se perde... >> escrita concorrente gera conflitos
 - Procedimentos de sincronismo de dados divergentes



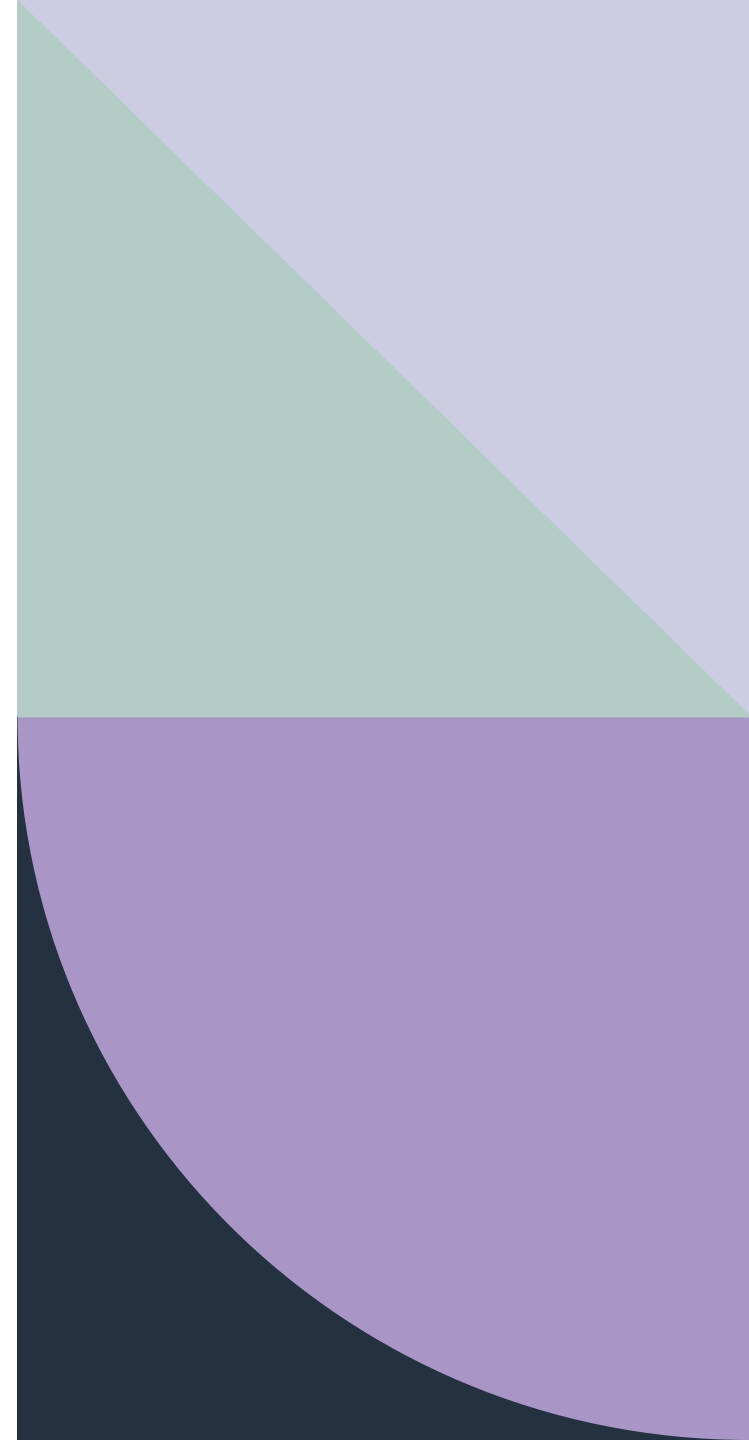
Bancos de Dados Chave-Valor

- Bancos de Dados Chave-Valor são sistemas extremamente simples e rápidos.
 - É como se o banco inteiro fosse uma única tabela com duas colunas:
 - Chave: chave primária
 - Valor: um tipo binário tipo BLOB ou similar, com o qual o sistema de banco de dados não tem nenhum controle
- Banco de dados NoSQL sem esquemas. São altamente particionáveis e possibilitam escalabilidade horizontal
 - Por exemplo, o Amazon DynamoDB alocará partições adicionais a uma tabela se uma partição existente for preenchida até o limite de capacidade e mais espaço de armazenamento for necessário.



Chave e Valor???

- Num Banco de Dados Chave-Valor, o VALOR é uma estrutura para armazenamento de dados que funciona de maneira similar a um MAPA ou DICIONÁRIO onde temos uma chave
- A CHAVE é o identificador para o registro





Implementações

- REDIS
- DynamoDB (Amazon)
- Voldemort (LinkedIn)
- RIAK





BANCO DE DADOS EM MEMÓRIA

Pontos positivos

- Quando há uma alta demanda por velocidade. Não fazem operações em disco

Pontos negativos

- PORÉM... a memória é volátil e se houver alguma falha elétrica, as informações podem ser comprometidas
- Ampliar a quantidade de memória RAM não é tão simples - compromete a escalabilidade



Armazenamento híbrido

- Informação persistida é armazenada em disco, mas parte dela é mantida em memória para acesso rápido



Bancos de Dados Chave-Valor – Onde usar???

- São SGBD simples, rápidos e úteis para compartilhamento de dados simples
 - Exemplos de aplicação
 - Dados de sessão de usuários (session storage)
 - Armazenamento de pedaços de HTML dinâmico pré-processados, evitando acessos a disco.
 - Carrinhos de compras: na temporada de compras de fim de ano, um site de comércio eletrônico pode receber bilhões de pedidos em segundos
 - PONTOS FORTES:
 - Pesquisas rápidas
 - FRAQUEZAS:
 - Dados armazenados sem esquema



Bancos de Dados Chave-Valor – Onde NÃO usar???

- Necessidade identificar relações entre os dados
- Operações de múltiplas chaves
- Requisitos de negócio mudam frequentemente



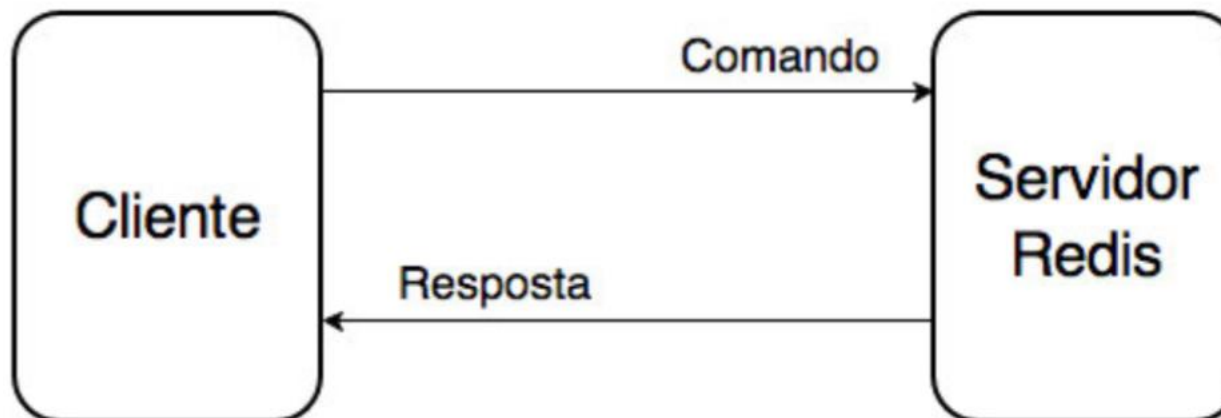
- Redis: *REmote DIctionary Server*
- Banco de dados *NoSQL*, open-source sob licença *BSD*
- Armazena seus dados em memória, embora seja possível persistir os dados fisicamente.
Extremamente rápido em operações de IO
- Todos os comandos executados no Redis são atômicos
Redis é executado como uma aplicação *single-thread* (enquanto um comando está sendo executado, nenhum outro comando será executado).
- Armazena os dados na forma de chave-valor
O valor contido na chave de um registro suporta diferentes formatos que podem ser strings, hashes, lists, sets e sets ordenados



- O Redis é um servidor TCP que faz uso do modelo cliente-servidor.

Uma requisição feita por um cliente ao servidor é seguida das seguintes etapas:

- O cliente envia um comando ao servidor e fica aguardando uma resposta do servidor (geralmente bloqueando a conexão) através de uma conexão estabelecida via socket;
- O servidor processa o comando e envia a resposta de volta ao cliente.





- O Redis oferece uma interface de linha de comando (CLI)
É a forma mais rápida e fácil de executar comandos Redis, mas normalmente a comunicação é feita via aplicação
Suporte a várias linguagens de programação: <http://redis.io/clients>

Redis em JAVA:

- configuração: programando com o Redis:

```
<dependency>  
  <groupId>redis.clients</groupId>  
  <artifactId>jedis</artifactId>  
  <version>2.4.2</version>  
</dependency>
```

```
Jedis jedis = new Jedis("localhost");  
String resultado = jedis.echo("ola redis!");  
System.out.println(resultado);
```



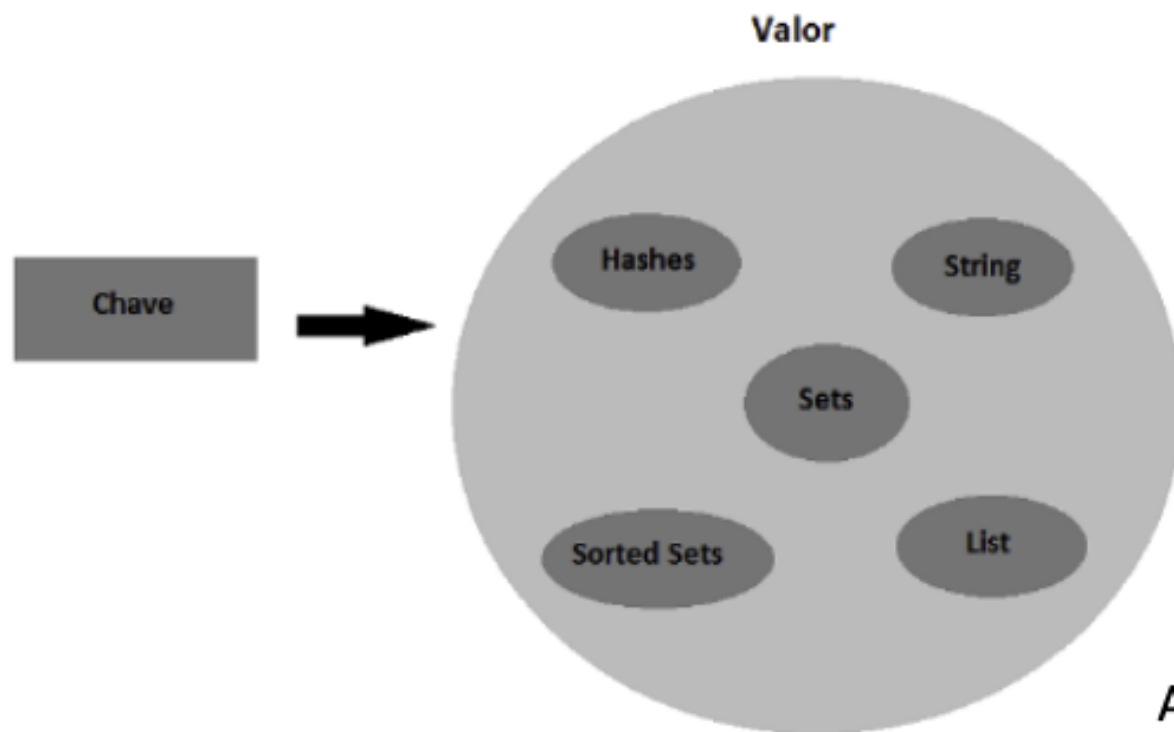
Testes online

- <http://try.redis.io> - Aplicação web
Disponibiliza um cliente desenvolvida em Ruby para testes do Redis





Redis

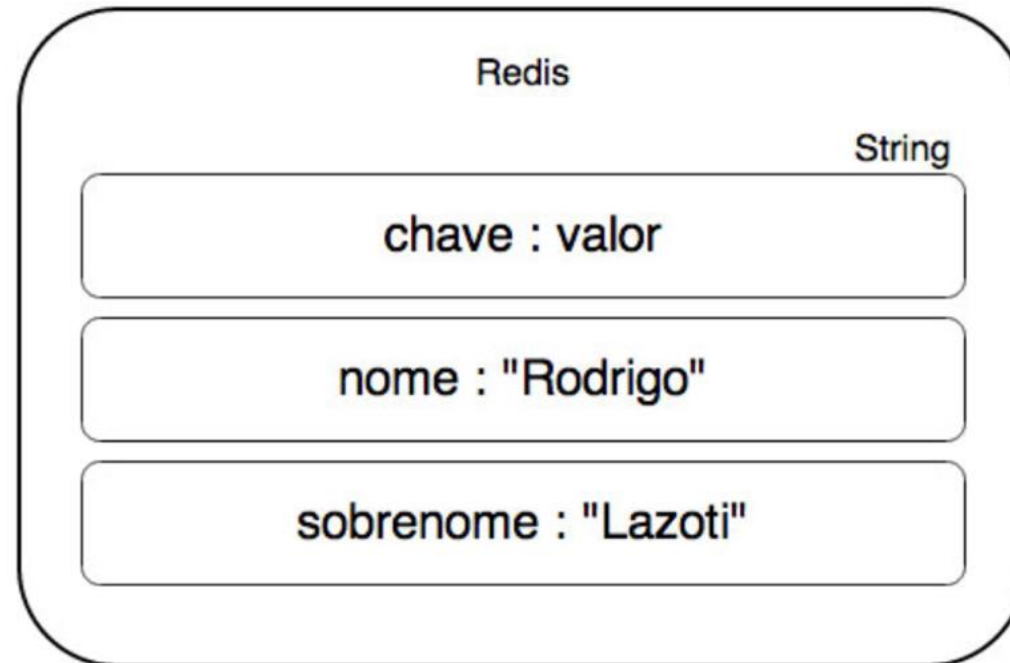


Adaptada de DAS 2015,



Redis - Cache de dados com strings

- String é o tipo de dado mais comum disponível no Redis
- Um valor do tipo string pode conter um tamanho de no máximo 512 Megabytes
Pode armazenar um texto, um JSON, objetos serializados ou até mesmo os dados binários de uma imagem





Redis - operações de Leitura e escrita

- Cache: repositório de dados em memória
- Comando GET/SET - Ler e armazenar dados no REDIS

```
public class ArmazenarUltimoUsuarioLogado {  
    public static void main(String[] args) {  
  
        Jedis jedis = new Jedis("localhost");  
        String resultado = jedis.set("ultimo_usuario_logado", "Tony Stark");  
        System.out.println(resultado);  
    }  
}
```

```
public class ObterUltimoUsuarioLogado {  
    public static void main(String[] args) {  
  
        Jedis jedis = new Jedis("localhost");  
        String valor = jedis.get("ultimo_usuario_logado");  
  
        System.out.println(valor);  
    }  
}
```



Redis - Chaves

- **tipo-de-objeto:identificador:nome-campo**
- Exemplo: armazenando sorteios da loteria no Redis

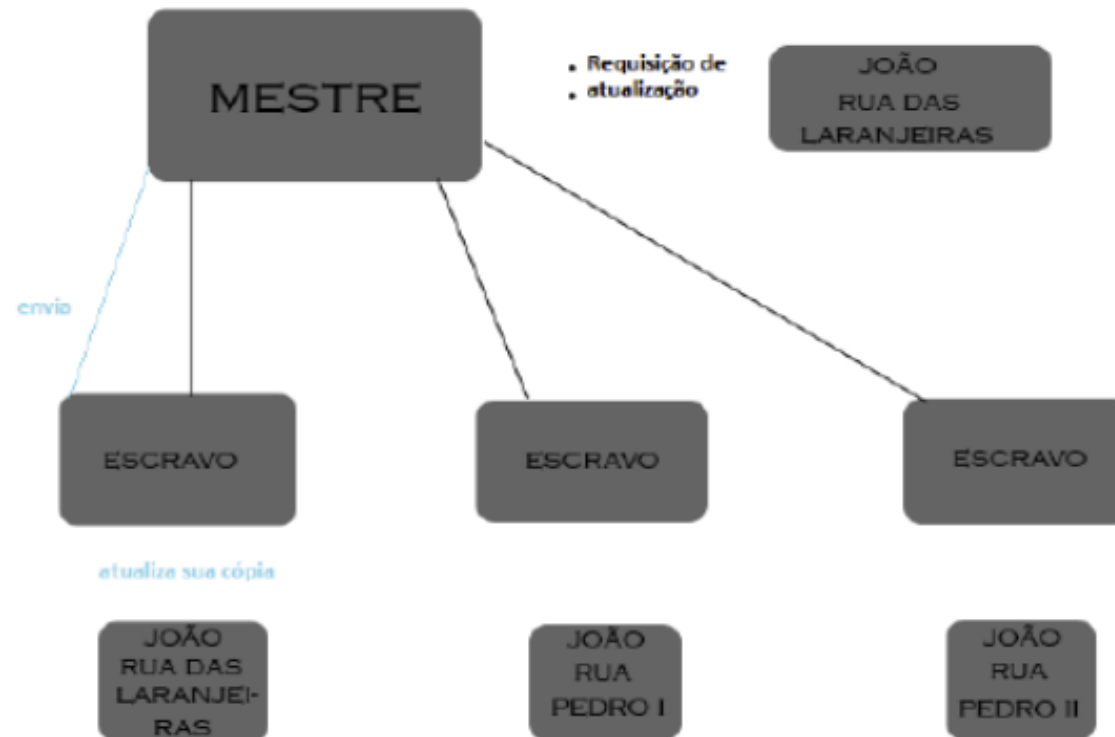
```
String dataDoSorteio1 = "04-09-2013";  
String numerosDoSorteio1 = "10, 11, 18, 42, 55, 56";  
String chave1 = String.format("resultado:%s:megasena", dataDoSorteio1);  
...  
Jedis jedis = new Jedis("localhost");  
String resultado = jedis.mset(  
    chave1, numerosDoSorteio1,  
    chave2, numerosDoSorteio2,  
    chave3, numerosDoSorteio3,  
    chave4, numerosDoSorteio4  
);
```

- MSET vs SET: MSET aceita vários conjuntos de chave-valor como parâmetro, SET aceita apenas um único conjunto de chave-valor.
 - "resultado:04-09-2013:megasena"



Disponibilidade do REDIS

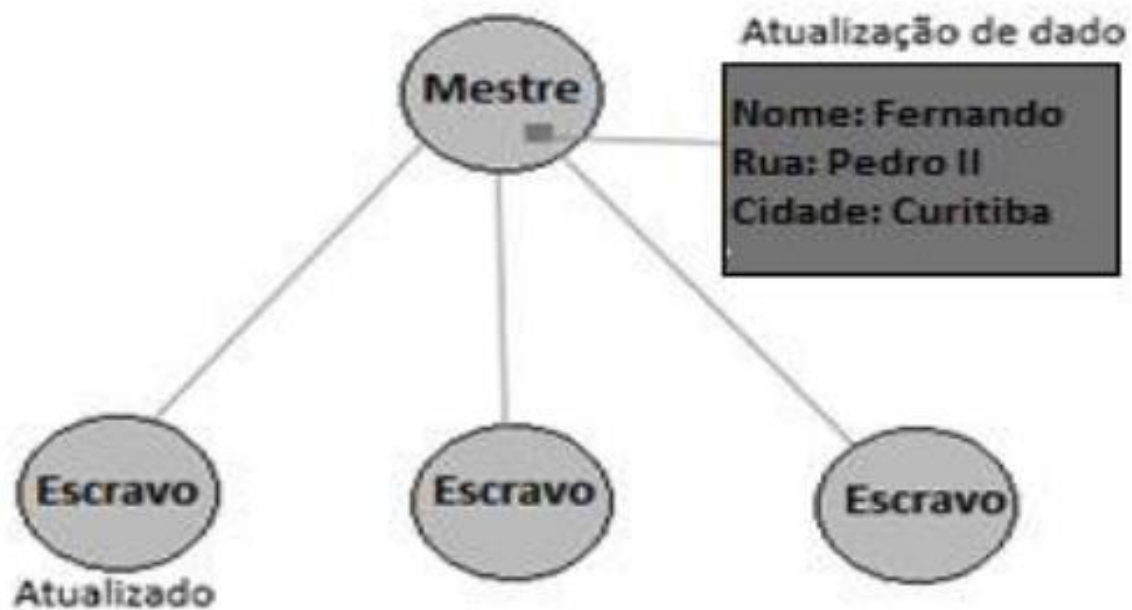
- Replicação de dados para aumentar a disponibilidade com arquitetura Master-Slave





Consistência no REDIS

- Consistência EVENTUAL





Tolerância à Partição no REDIS

- O Redis consegue garantir esta propriedade, por meio do uso da configuração Redis Sentinel.